Mock 세로 슬라이스 개발 기록

프로젝트: 빙의했더니 영지 멸망 3년 전

기록 기준일: 2026년 10월 30일

개발 범위: Godot와 ASP.NET Core, SQLite를 연결한 Act 1~3 Mock 흐름

1. 문서 목적

이 문서는 최초 개발 환경 구성부터 Mock 세로 슬라이스 완성까지의 기술적 판단, 구현 과정, 검증 근거와 문제 해결 경험을 기록한다. 단순 작업 일지가 아니라 향후 기능 확장과 포트폴리오 작성에 재사용할 수 있는 근거 문서를 목표로 한다.

2. 프로젝트 목표와 첫 번째 위험

최종 게임은 자유 대화, 고정 선택지, 영지 상태 계산, NPC 관계, 농사 미니게임, 제한시간 위기와 직업 엔딩을 포함한다. 그러나 이 요소를 한꺼번에 만들면 클라이언트와 서버의 계약이 맞지 않는 문제를 늦게 발견할 가능성이 높다.

따라서 첫 목표를 다음 한 줄로 제한했다.

> 타이틀 → NPC 대화 1회 → 선택지 1회 → 결과 계산 → 고정 직업 엔딩

이 범위는 화면, HTTP JSON 통신, 서버 상태 머신, 데이터 저장, 규칙 적용과 재시작을 한 번에 검증할 수 있는 최소 단위다. 실제 OpenAI API와 그래픽 작업은 이 흐름이 안정된 뒤로 미뤘다.

3. 아키텍처 원칙

3.1 서버 권위 구조

클라이언트는 플레이어 입력, 선택 ID와 농사 행동 결과만 전송한다. 능력치 변화량, 농사 점수, 진행 가능 여부와 엔딩은 서버가 결정한다. 이 구조는 클라이언트 조작을 줄이고 향후 콘텐츠가 늘어도 규칙을 한곳에서 관리하게 한다.

3.2 Mock과 Live의 동일 계약

Mock 모드도 실제 API와 같은 JSON 계약을 사용한다. 향후 OpenAI Provider를 추가할 때 Godot 화면과 진행 로직을 바꾸지 않고 서버 내부 구현만 교체할 수 있다.

3.3 데이터 기반 선택 규칙

각 Act의 장면, 대사, 선택지, 상태 변화와 직업 적성 변화는 content/acts/act*_mock.json에 정의했다. 새로운 선택지를 추가할 때 핵심 C# 코드를 반복 수정하지 않는 기반이다.

3.4 이벤트 기록과 중복 방지

각 세션은 진행 상태와 함께 append-only 게임 이벤트를 저장한다. 대화와 선택 요청은 UUID 형식의 idempotencyKey를 요구한다. 같은 키가 재전송되면 이미 저장된 응답을 돌려주므로 네트워크 재시도로 상태가 두 번 변하지 않는다.

4. 개발 과정

4.1 서버 도메인과 SQLite

다음 데이터를 SQLite에 저장하도록 구성했다.

- GameSession: Act, 턴, 진행 단계, 핵심 상태, 선택 결과와 엔딩
- GameEvent: 요청 순서, 이벤트 유형, 응답 JSON, 중복 요청 키
- NpcState: 신뢰, 두려움, 의심, 대화 요약
- TraitScore: 플레이 성향 점수
- CareerAffinity: 직업 적성과 마지막 변경 Act

서버 시작 시 개발용 데이터베이스를 생성하고, 세션 복구 API로 현재 상태와 완료된 엔딩을 다시 가져올 수 있게 했다.

4.2 Mock 게임 규칙

초기 상태는 군사 20, 재정 30, 정통성 25, 봉인 준비 10이다. 에드윈과 한 번 대화한 뒤 세 선택지를 제공한다.

선택	핵심 변화	직업 적성 변화
기사단 재건	군사 +20, 재정 -10	전투 +8, 지도력 +3
봉화 수리	봉인 +20, 재정 -10	마법 +8, 지도력 +3
주민 대피로 정비	정통성 +20, 재정 -5	지도력 +8, 외교 +3

Act 1~2 선택 결과는 다음 Act를 반환하며 상태와 적성을 누적한다. Act 3에서는 마르타와 농사 준비 방향을 정한 뒤 장애물 제거, 씨앗 배치, 물 배분과 작업 방식을 제출한다. 서버는 장애물 1개당 10점, 올바른 씨앗 1칸당 20점으로 계산하고 선택한 방식에 맞는 상태와 적성을 적용한다. 현재 임시 엔딩은 농사 완료 뒤에서만 반환한다.

4.3 Godot 화면 흐름

한 개의 메인 씬 안에서 타이틀, 대화, 선택지, 농사와 엔딩 화면을 전환한다. 선택 응답에 nextAct가 있으면 NPC와 장면을 갱신하고, nextActivity가 있으면 농사 화면을 표시한다. 요청 중에는 입력과 버튼을 잠가 중복 제출을 막고, 엔딩의 다시 시작 버튼은 새 세션을 생성한다.

헤드리스 실행 인자 --smoke-test는 세 NPC의 대화·선택, 농사, 임시 엔딩과 재시작을 자동으로 두 회차 실행한다.

5. API 흐름

```
POST /api/v1/sessions
  ↓ sessionId와 에드윈 장면
POST /api/v1/sessions/{id}/dialogues
  ↓ Mock 대사와 서사 피드백
GET /api/v1/sessions/{id}/choices
  ↓ 선택지 3개
POST /api/v1/sessions/{id}/choices
  ↓ 적용 변화, 다음 Act 또는 농사 활동
POST /api/v1/sessions/{id}/farming
  ↓ 서버 점수·보상 계산, 현재 임시 엔딩
GET /api/v1/sessions/{id}
  ↓ 재실행 후 현재 세션 복구
```

오류 응답은 code, message, retryable, requestId를 가진 공통 JSON 구조를 사용한다.

6. 검증 결과

6.1 빌드와 실행

- .NET 서버 빌드: 경고 0, 오류 0
- Godot 4.5.1 프로젝트 파싱과 실행 성공
- /health: 서버와 SQLite 연결 정상

6.2 정상 경로

- Act 1~2 선택 후 엔딩 없이 다음 Act로 전환하고 누적 상태 유지
- Act 3 준비 선택 후 농사 활동으로 전환, 최대 작업 결과 90점 계산
- 물 배분과 작업 방식에 따른 상태·적성 보상 적용
- Godot 클라이언트에서 세 Act, 농사, 임시 엔딩과 재시작을 두 회차 연속 완주
- 기사단 선택 후 군사 20에서 40으로 변경
- 봉화 선택 후 봉인 10에서 30으로 변경
- 대피로 선택 후 정통성 25에서 45로 변경

6.3 안정성 경로

- 같은 idempotencyKey 재전송 시 동일 응답 반환 및 중복 상태 변화 없음
- 허용 범위를 벗어난 농사 행동 횟수는 INVALID_FARMING_RESULT로 거부
- 빈 자유 대화는 HTTP 400과 INVALID_TEXT로 거부
- 서버 프로세스 재시작 후 SQLite에서 완료 세션과 엔딩 복구
- game.db, bin, obj, .godot은 Git 제외 확인
- 실제 API 키, GitHub 토큰과 사용자 대화 데이터가 소스에 없음

7. 문제와 해결

7.1 실행 환경의 로컬 포트 제한

자동화 환경 안에서는 로컬 포트 접근이 분리돼 서버와 클라이언트가 서로 보이지 않았다. 실제 로컬 실행 권한이 있는 환경에서 서버와 Godot을 실행해 종단간 통신을 검증했다. 코드 문제와 실행 격리 문제를 분리해 진단한 경험이다.

7.2 SQLite 네이티브 의존성 보안 경고

최초 복원에서 SQLitePCLRaw.lib.e_sqlite3 2.1.11에 대한 고위험 취약점 경고가 발생했다. 공식 보안 권고를 확인하고 SQLite 3.53.4 이상을 포함하는 SQLitePCLRaw.bundle_e_sqlite3 3.0.5를 직접 참조했다. 재빌드 결과 경고 0, 오류 0을 확인했다.

참고 자료:

- https://github.com/advisories/GHSA-2m69-gcr7-jv3q
- https://www.nuget.org/packages/SQLitePCLRaw.bundle_e_sqlite3

7.3 네트워크 재시도의 이중 적용 위험

진행 요청이 성공했지만 응답이 끊기면 클라이언트가 같은 요청을 재전송할 수 있다. 이벤트에 중복 요청 키와 전체 응답을 저장하고 같은 키에는 저장된 응답을 반환하도록 해 상태 변화가 한 번만 적용되게 했다.

8. 결과와 의미

이 작업으로 Godot UI, ASP.NET Core API, JSON 콘텐츠, SQLite 저장과 서버 규칙이 세 Act와 농사 활동에 걸쳐 연결되는 것을 확인했다. 대화·선택 외의 조작 결과도 서버가 검증하는 기반을 만들었으며 Act 4 제한시간 선택도 같은 책임 분리를 유지할 수 있다.

포트폴리오 관점에서 다음 역량을 증명하는 작업이다.

- 큰 기획을 가장 위험한 종단간 흐름으로 축소하는 범위 설계
- 클라이언트와 서버 책임 분리
- 데이터 기반 규칙 설계
- SQLite 기반 상태 및 이벤트 저장
- 중복 요청과 오류 계약을 고려한 API 안정성
- 실제 Godot 클라이언트를 포함한 종단간 자동 검증
- 의존성 취약점 탐지와 해결

9. 한계와 다음 작업

현재 구현은 Act 1~3과 농사 뒤의 임시 직업 엔딩으로 제한된다. 다음 단계에서는 AI를 연결하기 전에 Mock 상태로 아래 범위를 완성한다.

1. Act 4 균열 발생과 20초 최종 선택
2. 누적 상태 기반 영지 운명과 직업 위상 판정
3. 이어하기 UI와 주요 결과 경로 자동 테스트

전체 Mock 게임이 안정적으로 완주된 뒤에만 OpenAI API Provider와 생성형 후일담을 추가한다.

10. 재현 방법

서버 실행:

```
dotnet run --project server/Game.Api/Game.Api.csproj
```

Godot 실행:

```
/Applications/Godot.app/Contents/MacOS/Godot --path client
```

자동 스모크 테스트:

```
/Applications/Godot.app/Contents/MacOS/Godot --headless --path client -- --smoke-test
```

PDF 재생성:

```
python3 tools/generate_development_report.py
```